

Problem Set 4

tianzheyu submissions

tianzheyu submissions							
#	When	Who	Problem	Lang	Verdict	Time	Memory
379903807	Jun/23/2026 22:06 UTC+10	tianzheyu	C - Coloring Trees	C++23 (GCC 14-64, msys2)	Accepted	203 ms	5200 KB

<https://codeforces.com/submissions/tianzheyu#>

C. Coloring Trees

Process

The question gives n trees, each has a color value and how many paint we need to color the tree for each color. And we have m colors in total. For each tree, if its color value $c = 0$, this means we can paint any color to it with the cost we've mentioned. And the question defines the beauty of the coloring as the minimum number of contiguous groups (each group contains some subsegment of trees) you can split all the n trees into so that each group contains trees of the same color. The task is to find the minimum paint require to get beauty of the coloring equal to k .

Challenges and Overcoming

If we calculate the paint needed in the order of tree, for each tree we need to know the previous tree's color and the previous beauty of the coloring. This means we need a 3D dp table ($dp[n][k][m]$), where n keeps track of the current tree we are calculating, k keeps track of the beauty of the coloring we previously achieved and m keeps track of which color is it for the previous tree.

Base Cases

For the first tree ($i = 1$), the beauty of the coloring must be 1 since it's the start of the sequence. And we initialize the rest of cells $1e18$ to prevent the $\min()$ function from incorrectly picking 0 or -1.

1. If the first tree is already colored and the current color is equal to the first tree color: It requires no paint, so $dp[1][1][color] = 0$.
2. If the first tree is uncolored: We can paint it any color q from 1 to m . The cost will be the paint cost for that color: $dp[1][1][q] = paints[1][q]$.

Transform Function

For tree i from 2 to n , target beauty j from 1 to k , and current color q from 1 to m :

1. **If the current tree is already colored:** We can only transition to its specific color q (cost is 0). We check the previous tree's color p :
 - o If $p == q$, the beauty stays the same:
 $dp[i][j][q] = \min(dp[i][j][q], dp[i-1][j][p])$
 - o If $p != q$, the beauty increases by 1:
 $dp[i][j][q] = \min(dp[i][j][q], dp[i-1][j-1][p])$

2. **If the current tree is uncolored:** We iterate through all possible colors q to paint it, adding `paints[i][q]` to the cost. We also check the previous tree's color p :

- If $p == q$, the beauty stays the same:

`dp[i][j][q] = min(dp[i][j][q], dp[i - 1][j][p] + paints[i][q])`

- If $p != q$, the beauty increases by 1:

`dp[i][j][q] = min(dp[i][j][q], dp[i - 1][j - 1][p] + paints[i][q])`

Final Answer

We iterate through all possible colors q from 1 to m for the last tree n at exactly beauty k , and find the minimum value: `result = min(dp[n][k][q])`

If the result stays `1e18` then it means it is impossible to achieve beauty k therefore, we put -1 into the result.